# Technical Design Document: Gated Township Resident Management System

**Repository Status**: Local Git Repository Initialized ( `branch: main` , verified commit `6c241318c093a20dfc60464289e9dd3114b1092a` )
**GitHub URL**: Pending remote publication (commands provided below)
**Live Application Status**: Local & Docker deployment ready ( `http://localhost:8080` & `http://localhost:5173` )
**Target Township**: Pune Grand Vista, Baner-Balewadi, Pune, Maharashtra (28 Towers • 2,400 Apartments)
**Author**: Engineering Architecture Team
**Date**: November 8, 2026
**Document Version**: 1.0.0 (Phase 12 Final Release)

---

## 1. Project Overview & Client Problem Statement

Large residential gated townships in India (such as Pune Grand Vista, comprising 2,400 residential flats across 28 multi-story towers) face severe operational, financial, and safety bottlenecks when managed via fragmented spreadsheets, paper registers, and generic messaging groups:

1. **Maintenance Billing & Arrears Opacity**: Society bye-laws require uniform calculation based on carpet area (INR 6.50/sq ft/month), strict due dates, late fees, and immutable double-entry ledger bookkeeping. Manual spreadsheets frequently create reconciliation disputes between flat owners and the managing committee.
2. **Payment Reconciliation Inefficiencies**: High-volume NEFT, RTGS, and UPI bank transfers arrive with truncated bank narrations, leaving staff unable to correlate transactions to specific apartment numbers without manual investigation.
3. **Emergency & Safety Hazards in Complaints**: Standard ticketing systems treat life-threatening incidents (gas leaks, elevator power failures with passengers trapped, sparking transformer boxes) with the same priority as minor cosmetic tickets, risking human life and statutory liability.
4. **Gate Security Vulnerabilities**: Over 1,000 daily delivery couriers, cabs, and visitors create chaotic security cabin queues. Manual paper visitor books compromise resident privacy, cannot be audited, and fail during internet connectivity outages.
5. **Clubhouse & Amenity Clashes**: Unregulated access to shared swimming pools, gyms, and community halls results in overcrowding and duplicate reservations.
6. **Executive RWA Governance Gaps**: Managing Committees require real-time visibility into collection rates, dues ageing buckets, and SLA compliance without exposing sensitive resident contact credentials.

The **Gated Township Resident Management System** resolves these challenges through a unified, secure, multi-persona web application and REST API.

---

## 2. Architecture & Technology Choices

```
graph TB
    subgraph ClientLayer["Frontend Client Layer"]
        SPA["React 19 SPA (Vite 8 + Tailwind CSS v4)"]
        Router["React Router v7"]
        State["Auth Context + Local Storage Sync"]
    end

    subgraph SecurityLayer["Security & Gateway Layer"]
        Nginx["Nginx Reverse Proxy (SSL / Gzip / Caching)"]
        SecurityFilter["Spring Security 6.x JWT Authentication Filter"]
        RBAC["Method Security (@PreAuthorize)"]
    end

    subgraph BackendLayer["Spring Boot 4 Application Layer"]
        AuthSvc["Auth & User Service"]
        BillSvc["Maintenance Billing & Ledger Engine"]
        PaySvc["Payment & Reconciliation Engine"]
        AiSvc["AI Complaint Classifier (Gemini / Rules)"]
        GateSvc["Visitor & Gate Pass Engine"]
        AmenitySvc["Amenity Booking Engine"]
        NoticeSvc["Broadcast Notice Engine"]
        AnalyticsSvc["Committee Analytics Engine"]
    end

    subgraph StorageLayer["Data & Persistence Layer"]
        Postgres[(PostgreSQL 16 Engine)]
        Hikari["Hikari Connection Pool (ACID Isolation)"]
        DiskStore["Hardened Local Media Storage"]
    end

    SPA --> Nginx
    Nginx --> SecurityFilter
    SecurityFilter --> RBAC
    RBAC --> BackendLayer
    BackendLayer --> Hikari --> Postgres
    BackendLayer --> DiskStore
```

### Technology Decision Rationale

- **Java 17 LTS & Spring Boot**: Chosen for enterprise-grade transactional boundaries (`@Transactional`), mature ORM with Spring Data JPA/Hibernate, robust security filtering, and non-blocking background concurrency.
- **PostgreSQL 16**: Selected for ACID transaction isolation, relational foreign keys, unique constraint guarantees across towers/flats/users, and high-performance financial aggregation queries.
- **React 19 & Vite 8**: Ensures sub-second local build times, tree-shaking, and high-performance client rendering with responsive Tailwind CSS.

- **Stateless JWT (HMAC-SHA512)**: Enables horizontally scalable REST authentication without server-side session state, with user claims and roles signed cryptographically.
- **Zero Floating-Point Financials**: `java.math.BigDecimal` with `RoundingMode.HALF_UP` is enforced across all currency fields to eliminate IEEE 754 precision drift.

---

## 3. Database Model & Major Entities

The relational schema is structured into 8 modular Flyway migration stages:

```
erDiagram
    TOWERS ||--o{ FLATS : contains
    FLATS ||--o{ FLAT_MEMBERS : houses
    USERS ||--o{ FLAT_MEMBERS : belongs_to
    FLATS ||--o{ MAINTENANCE_BILLS : billed_to
    FLATS ||--o{ FLAT_LEDGER_ENTRIES : tracks
    MAINTENANCE_BILLS ||--o{ PAYMENTS : settles
    FLATS ||--o{ COMPLAINTS : reports
    USERS ||--o{ COMPLAINTS : files
    COMPLAINTS ||--o{ COMPLAINT_ACTIVITIES : audits
    FLATS ||--o{ VISITORS : hosts
    VISITORS ||--o{ VISITOR_ACTIVITIES : audits
    AMENITIES ||--o{ AMENITY_BOOKINGS : reserves
    FLATS ||--o{ AMENITY_BOOKINGS : books
    TOWERS ||--o{ NOTICES : targets
    USERS ||--o{ NOTICE_READS : acknowledges
```

### Key Relational Entities

1. **`towers`**: Represents the 28 residential high-rises (e.g. `T01` to `T28`), storing floor counts and unit allocations.
2. **`flats`**: Represents individual units, enforcing unique `(tower_id, flat_number)` pairs, floor numbers, carpet area in sq ft, and occupancy status.
3. **`users`**: Encapsulates residents, facility staff, guards, and committee members with BCrypt hashed passwords, unique emails, unique mobile phones, and role constraints (`RESIDENT`, `HELP_DESK`, `GUARD`, `COMMITTEE`).
4. **`flat_members`**: Linking table implementing flat-level resident authorization (`PRIMARY_OWNER`, `TENANT`, `FAMILY_MEMBER`).
5. **`maintenance_bills`**: Stores monthly billing records calculated at INR 6.50/sq ft, tracking `total_due`, `paid_amount`, `outstanding_balance`, and status (`UNPAID`, `PARTIALLY_PAID`, `PAID`, `OVERDUE`).
6. **`flat_ledger_entries`**: Immutable financial ledger tracking debit invocations, credit payments, and running balance.
7. **`bank_transactions`** & **`bank_statement_batches`**: Captures uploaded bank CSV entries, matching UTRs and routing ambiguous narrations to manual staff review.

8. `complaints` & `complaint_activities`: Full ticketing lifecycle tracking category, priority, AI suggestions, safety escalation flags, SLAs, and immutable audit logs.
9. `visitors` & `visitor_activities`: Pre-approved and walk-in gate passes with QR tokens, valid time windows, photos, and guard verification timestamps.
10. `amenities` & `amenity_bookings`: Manages clubhouse assets (Gym, Pool, Badminton, Hall) with slot durations, operating hours, capacity limits, and duplicate prevention.
11. `notices` & `notice_reads`: Society announcements targeted township-wide or by tower, with pinning priority and read receipts.

---

## 4. Main User Flows

```
sequenceDiagram
    autonumber
    actor Resident
    actor Guard
    actor HelpDesk
    actor Committee
    participant Gateway as Spring Security
    participant App as Backend Service
    participant DB as PostgreSQL 16

    Note over Resident,DB: 1. Resident Bill Payment Flow
    Resident->>Gateway: POST /api/payments/test (Bill ID, Amount)
    Gateway->>App: Validate JWT & Flat Authorization
    App->>DB: Check Bill Balance & Prevent Overpayment
    App->>DB: Record Payment & Append Immutable Credit to Ledger
    App->>Resident: Return 200 OK with Verified Receipt

    Note over Resident,Guard: 2. Gate Security & Visitor Pass Flow
    Resident->>App: Pre-Approve Visitor (Name, Phone, Time)
    App->>DB: Persist Visitor Pass & Generate Secure 6-Digit Pass Code
    App->>Resident: Display Shareable Pass with QR Token
    Guard->>App: Lookup Pass Code at Gate Cabin
    App->>DB: Verify Valid Window & Check In Visitor
    App->>Guard: Confirm Entry & Print Visitor Pass

    Note over HelpDesk,DB: 3. Bank CSV Statement Reconciliation
    HelpDesk->>App: Upload Bank Statement CSV
    App->>App: Parse UTR & Execute 4-Tier Match Engine
    App->>DB: Persist Matches (Auto Matched vs Manual Review)
    HelpDesk->>App: Approve Matched Transaction
    App->>DB: Update Bill & Post Credit to Flat Ledger

    Note over Committee,DB: 4. Executive Analytics & Reporting
    Committee->>Gateway: GET /api/committee/dashboard
    Gateway->>App: Verify ROLE_COMMITTEE / ROLE_HELP_DESK
    App->>DB: Aggregate Real PostgreSQL Dues, Ageing & SLAs
    App->>Committee: Deliver Financial KPIs, Ageing Buckets & CSV Exports
```

## 5. AI Complaint Classification, Safety Overrides & Fallback Engine

The system incorporates a hybrid AI classification architecture designed for maximum reliability and life safety:

```
flowchart TD
    Start["Resident Submits Complaint (Title + Description)"] --> Step1["Scan for Life-Safety Trig
    Step1 --> CheckSafety{"Safety Trigger Detected?<br/>(Gas leak, fire, lift trap, live wire)"}

    CheckSafety -->|YES| Override["CRITICAL SAFETY ESCALATION<br/>Enforce Priority: EMERGENCY<br/>
    CheckSafety -->|NO| CheckGemini{"GEMINI_API_KEY Configured?"}

    CheckGemini -->|YES| GeminiAI["Query Google Gemini API (Interactions)<br/>Analyze context & cl
    CheckGemini -->|NO / Timeout| RuleEngine["Execute Certified Semantic Rule Engine<br/>Keyword s

    GeminiAI --> Output["Generate AI Suggestions (Category, Priority, Confidence Score)"]
    RuleEngine --> Output
    Override --> Save["Persist Complaint & Create Activity Audit Record"]
    Output --> Save
```

### Safety Rules & Guarantee

- **Keywords**: "gas leak", "lpg smell", "cylinder leak", "elevator trapped", "lift stuck", "lift failure", "fire", "smoke", "sparking", "electric shock", "exposed wire", "short circuit", "water main burst", "flooding", "ceiling collapse".
- **Enforcement**: When any safety keyword is matched in title or description, `isCriticalSafetyEscalation` is permanently set to `true`, priority is overridden to `EMERGENCY`, and response deadlines are tightened to 2 hours.
- **Safety Downgrade Prevention**: Once escalated, the safety flag cannot be cleared or downgraded by subsequent ticket updates.
- **Zero-Dependency Fallback**: If internet connectivity is interrupted or no external API key is supplied, the local semantic rule engine runs deterministically in 0ms, guaranteeing 100% classification availability.

---

## 6. Key UI/UX Decisions

1. **Enterprise SaaS Visual Design**:
   - Palette inspired by modern enterprise software: `#0c1322` deep navy blue sidebar, crisp `#ffffff` canvas, subtle `#0d9488` teal accents, and `#64748b` slate typography.
   - High-contrast visual hierarchy with GitHub-style status badges.
2. **Responsive Mobile-First Gate Interface**:
   - Gate guards use touchscreen tablets with extra-large tap targets (48px+), fast numeric keypad inputs, high-contrast pass status badges, and instant QR camera scanning.
3. **Resilient Offline Mode for Gate Cabin**:
   - Offline actions are persisted in browser `localStorage`. When the guard tablet recovers connectivity, an **Offline Sync** badge alerts the guard to flush the queue into PostgreSQL with zero data loss.

4. **Role-Aware Contextual Header**:
    - Dynamically highlights active persona (*Resident*, *Help Desk*, *Security Gate*, *Managing Committee*) with one-click access to demo accounts in development environments.

---

## 7. Security & Privacy Decisions

1. **Public Registration Role Hardening**:
    - Public registration endpoints ( `/api/auth/register` ) unconditionally assign `ROLE_RESIDENT` . Privileged roles ( `HELP_DESK` , `GUARD` , `COMMITTEE` ) can only be provisioned through internal administrative seeding.
2. **Resident Flat-Level Data Isolation**:
    - All resident endpoints strictly enforce `flatMemberRepository.existsByFlatIdAndUserId()` . Residents attempting to inspect bills, ledgers, or visitor passes belonging to other units receive HTTP 403 Forbidden.
3. **Statutory Defaulter Privacy Protection**:
    - In accordance with co-operative society bye-laws, defaulter rosters omit personal telephone numbers, personal emails, and login credentials, exposing only tower ID, flat number, carpet area, days overdue, and outstanding arrears.
4. **Guard Role Restriction**:
    - Guards are prohibited from approving visitor passes on behalf of residents ( `VisitorService.processResidentApproval` explicitly blocks `Role.GUARD` ).
5. **Bank Ambiguity Shield**:
    - Ambiguous bank statement line items are never auto-posted to ledgers. Transactions with matching amounts but ambiguous narrations are routed to `MANUAL_REVIEW` or `UNMATCHED` .

---

## 8. Assumptions & Design Constraints

1. **Society Maintenance Tariff**: Flat monthly rate of INR 6.50 per square foot based on flat carpet area as established in the original township brief.
2. **Payment Processing**: Mock payment gateway in test mode simulating full and partial settlements, UTR generation, and immutable ledger crediting without third-party banking fees.
3. **Gate Security Hardware**: Camera scanning utilizes standard HTML5 video capture APIs and browser QR scanners; automated boom barrier hardware integration requires an MQTT/GPIO relay controller.
4. **Township Size**: Initialized for 28 residential towers (T01 through T28) and 2,400 apartments in Pune, Maharashtra.

---

## 9. Features Deliberately Cut from Scope

To ensure architectural focus, system stability, and timely delivery of core township management, the following were deliberately deferred:

- Direct real-money payment gateway integration (Razorpay / PayU API merchant credentials).
- Physical hardware boom barrier serial/relay drivers.
- Native mobile apps (React Native / Flutter); responsive mobile web SPA serves all devices.
- SMS / WhatsApp OTP gateway APIs (mocked in-app via direct credentials for reliable testing).

---

## 10. Known Limitations & Future Improvements

| Limitation | Impact | Recommended Production Roadmap |
|---|---|---|
| **Local Photo Storage** | Uploaded photos are stored on local filesystem ( `/uploads` ). | Migrate storage provider to AWS S3 or Google Cloud Storage with pre-signed URLs. |
| **Single-Node In-Memory Scheduler** | Timed jobs run inside the Spring Boot container. | Introduce Quartz or ShedLock backed by PostgreSQL for multi-node clustering. |
| **Email Delivery** | Circulars and notices are tracked in-app. | Integrate SendGrid or AWS SES for automated email digest broadcasts. |
| **Automated Meter Reading (AMR)** | Water tanker supplementary calculations are manual. | Integrate IoT water meters via LoRaWAN / MQTT telemetry into billing. |